HostelHub

End-to-End System Documentation

Complete Architecture, Database Schema, REST API Reference & Socket.io WebSockets Guide

PROJECT NAME
HostelHub Management System

DOCUMENT VERSION
v2.5.0 (Production Release)

BACKEND TECH
Node.js, Express, Sequelize, PostgreSQL, Socket.io

FRONTEND TECH
Angular Standalone, TypeScript, Capacitor Android

AUTHOR ENGINEERING TEAM
Abhinav Kumar (Lead Full-Stack Developer)

DATE GENERATED
August 2026

📖 Table of Contents

1. Executive Summary & Architecture Overview

HostelHub is a full-stack digital hostel management platform engineered to automate maintenance complaint dispatching, daily student roll call attendance, mess menu regulations, meal skipping, official announcements, and real-time batch group communications.

> **Key Architectural Design:** The system features a decoupled Client-Server architecture. The frontend is built as a single-page progressive web application using **Angular (Standalone Components)** and compiled into a native Android APK via **Capacitor**. The backend runs on a high-throughput **Node.js / Express.js REST API** layer paired with **Sequelize ORM** and **PostgreSQL**. Real-time updates (chat messages, ticket updates, notifications) are powered by **Socket.io**.

ROLE	PORTAL ACCESS	KEY RESPONSIBILITIES
Student	Student Portal	Raise complaints with photos, track ticket progress, view mess menu, skip meals, give mess feedback, view attendance metrics, join hostel/batch group chats, edit profile.
Warden	Warden Portal	Approve new student registrations, assign maintenance staff to complaints, mark daily roll call attendance, publish announcements, monitor mess skip stats, respond to warden group chats.
Staff	Staff Portal	View assigned maintenance jobs, update ticket status (In Progress), upload job completion proof photo, submit resolved status.
Admin	Admin Portal	Manage system users, create warden/staff accounts, view global analytics, configure public footer & developer settings, manage database overrides.

2. Core Tech Stack & Dependencies

LAYER	TECHNOLOGY	VERSION / PURPOSE
Frontend Framework	Angular	v19 (Standalone Architecture, RxJS, HttpClient, FormsModule)
Mobile Runtime	Capacitor	v8 (Native Android Bridge, Camera API, Preferences, App Back Button Listener)
Design System	Vanilla CSS Tokens	Custom Design Tokens, Dynamic Light/Dark Theme, Glassmorphism, Micro-animations
Backend Environment	Node.js & Express.js	RESTful Routing, Async Controllers, Middleware Stack
Database & ORM	PostgreSQL & Sequelize	Relational Schema, Foreign Key Constraints, Transactions, Auto-migrations
Real-Time Engine	Socket.io	Bidirectional WebSockets for Chat, Notifications, Ticket & Announcement Sync
Storage Layer	Supabase / Local Disk Fallback	Cloud Image Storage for complaint attachments, completion proofs, profile pictures
Authentication	JWT & bcryptjs	JsonWebToken bearer authentication, bcrypt password hashing, Role-Based Access Control

3. Database ERD & Entity Specifications

3.1 User Model (users table)

Stores credentials, personal details, hostel assignment, roll number, and approval flags.

```
User {
  id: INTEGER (PK, AutoIncrement)
  name: STRING (NotNull)
  email: STRING (NotNull, Unique)
  password: STRING (NotNull)
  role: ENUM('student', 'warden', 'staff', 'admin') (Default: 'student')
  hostelBlock: STRING ('Boys Hostel 1', 'Boys Hostel 2', 'Girls Hostel 1', 'Girls Hostel 2')
  roomNumber: STRING
  rollNumber: STRING
  batch: STRING (e.g. 'Batch 2025')
  gender: STRING ('Male', 'Female', 'Other')
  phone: STRING
  bio: TEXT
  profilePicUrl: STRING
  status: ENUM('pending', 'active', 'rejected') (Default: 'pending')
  reApprovalStatus: BOOLEAN (Default: false)
  googleId: STRING (Nullable)
  createdAt: DATETIME
  updatedAt: DATETIME
}
```

3.2 Complaint Model (complaints table)

```
Complaint {
  id: INTEGER (PK, AutoIncrement)
  studentId: INTEGER (FK -> users.id)
  staffId: INTEGER (FK -> users.id, Nullable)
  title: STRING (NotNull)
  description: TEXT (NotNull)
  category: ENUM('electrical', 'plumbing', 'carpentry', 'cleaning', 'other')
  priority: ENUM('low', 'medium', 'high', 'urgent') (Default: 'medium')
  status: ENUM('pending', 'assigned', 'in_progress', 'resolved') (Default: 'pending')
  photoUrl: TEXT (Nullable)
  completionPhotoUrl: TEXT (Nullable)
  feedbackRating: INTEGER (Nullable, 1-5)
  feedbackComment: TEXT (Nullable)
  createdAt: DATETIME
  updatedAt: DATETIME
}
```

3.3 Mess Management Models

```
MessMenu {
  id: INTEGER (PK, AutoIncrement)
  dayOfWeek: ENUM('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday')
  breakfast: TEXT
  lunch: TEXT
  snacks: TEXT
  dinner: TEXT
  isSpecial: BOOLEAN (Default: false)
}

MessSkip {
  id: INTEGER (PK, AutoIncrement)
  studentId: INTEGER (FK -> users.id)
  date: DATEONLY (NotNull)
  mealType: ENUM('breakfast', 'lunch', 'snacks', 'dinner')
}

MessFeedback {
  id: INTEGER (PK, AutoIncrement)
  studentId: INTEGER (FK -> users.id)
  rating: INTEGER (1-5)
  category: STRING ('Quality', 'Hygiene', 'Quantity', 'Behavior')
  comment: TEXT
}
```

3.4 Attendance Model (attendances table)

```
Attendance {
  id: INTEGER (PK, AutoIncrement)
  studentId: INTEGER (FK -> users.id)
  date: DATEONLY (NotNull)
  status: ENUM('present', 'absent') (NotNull, Default: 'present')
  markedBy: INTEGER (FK -> users.id)
  remarks: TEXT (Nullable)
}
```

3.5 GroupChat & ChatMessage Models

```
GroupChat {
  id: INTEGER (PK, AutoIncrement)
  name: STRING (NotNull)
  type: ENUM('block', 'batch', 'all_hostel', 'wardens')
  hostelBlock: STRING (Nullable)
  batch: STRING (Nullable)
}

ChatMessage {
  id: INTEGER (PK, AutoIncrement)
  groupId: INTEGER (FK -> group_chats.id)
  senderId: INTEGER (FK -> users.id)
  message: TEXT
  attachmentUrl: TEXT (Nullable)
  isDeleted: BOOLEAN (Default: false)
  deletedByName: STRING (Nullable)
  createdAt: DATETIME
}
```

4. Complete REST API Endpoints Reference

4.1 Authentication & Profile APIs (/api/auth)

POST `/api/auth/register`

Description: Register a new student or warden account. Requires Warden approval before login if role is student.

Body (JSON): `{ name, email, password, role, hostelBlock, roomNumber, rollNumber, batch, gender, phone }`

Response (201 Created): `{ message: "Registration successful! Pending Warden approval.", user: {...} }`

POST `/api/auth/login`

Description: Authenticates user credentials and returns JWT bearer token with user object.

Body (JSON): `{ email, password }`

Response (200 OK): `{ token: "eyJhbGci...", user: { id, name, email, role, status, ... } }`

GET `/api/auth/profile`

Headers: Authorization: Bearer <token>

Response (200 OK): Returns authenticated user profile object.

`PUT` **`/api/auth/profile`**

Description: Updates user profile. If student changes **gender** or **batch**, `reApprovalStatus` is set to true and Warden approval is required.

Content-Type: multipart/form-data (optional `profilePic` file upload)

`POST` **`/api/auth/forgot-password`**

Description: Generates 6-digit OTP and sends email verification code for password reset.

4.2 User Management & Approvals APIs (/api/users)

`GET` **`/api/users/pending`**

Access: Warden, Admin

Description: Retrieves list of pending student registrations and critical profile edit re-approvals.

`PUT` **`/api/users/approve/:userId`**

Access: Warden, Admin

Description: Approves student account or critical profile edits, setting status to 'active' and reApprovalStatus to false.

`DELETE` **`/api/users/reject/:userId`**

Access: Warden, Admin

Description: Rejects pending registration and removes user account record.

4.3 Maintenance Complaints APIs (/api/complaints)

POST `/api/complaints/raise`

Access: Student

Payload: multipart/form-data with `title, description, category, priority, photo`

PUT `/api/complaints/assign/:complaintId`

Access: Warden

Body: `{ staffId: 5 }`

Description: Assigns service staff member to ticket and updates status to 'assigned'.

PUT `/api/complaints/update-status/:complaintId`

Access: Staff

Payload: multipart/form-data with `status ('in_progress' | 'resolved'), completionPhoto`

4.4 Mess Menu, Skipping & Feedback APIs (/api/mess)

GET `/api/mess/menu`

Description: Fetches 7-day mess menu schedule including Breakfast, Lunch, Evening Snacks, and Dinner.

PUT `/api/mess/menu/:id`

Access: Warden, Admin

Body: `{ breakfast, lunch, snacks, dinner, isSpecial }`

POST `/api/mess/skip`

Access: Student

Body: `{ date: "2026-08-06", mealType: "lunch" }`

4.5 Attendance APIs (/api/attendance)

POST `/api/attendance/mark`

Access: Warden, Admin

Body: `{ date: "2026-08-05", attendances: [{ studentId: 10, status: "present", remarks: "" }] }`

GET `/api/attendance/summary?date=2026-08-05&hostelBlock=Boys Hostel 1`

Access: Warden, Admin

Description: Returns filterable count totals: Total Students, Present Count, Absent Count.

4.6 Real-Time Group Chat APIs (/api/chat)

GET `/api/chat/groups`

Description: Fetches user's relevant group chats (Hostel Block Chat, Academic Batch Chat, All Hostel Group, Warden Admin Chat).

POST `/api/chat/send`

Body: `{ groupId: 2, message: "Hello team", attachmentUrl: null }`

5. Socket.io WebSockets Specifications

EVENT NAME	DIRECTION	PAYLOAD STRUCTURE	DESCRIPTION
`join_room`	Client → Server	`"group_2"` or `"user_10"`	Joins specified Socket room for live updates.
`send_message`	Client → Server	`{ groupId, senderId, message, attachmentUrl }`	Emits live chat message to room members.
`new_message`	Server → Client	`{ id, groupId, senderId, message, sender: {...} }`	Broadcasts message to room subscribers instantly.
`notification`	Server → Client	`{ message, type, createdAt }`	Triggers in-app toast notification.
`delete_message_everyone`	Server → Client	`{ messageId, groupId, deletedByName }`	Soft deletes message in real-time for all members.

6. Critical Enforcements & Security Rules

Profile Re-Approval Rule: When a student updates their profile details, editing standard fields (Name, Phone, Bio, Room Number, Profile Picture) is updated immediately. However, if the student changes **Gender** or **Academic Batch**, a security alert pops up warning that these are critical fields, setting `reApprovalStatus = true` requiring Warden re-approval.

Real-Time Roll Call: Daily Roll Call Attendance strictly tracks binary statuses (**Present** vs **Absent**) with hostel-wise filtering and bulk "Mark All Present" capabilities.